3. Architect a Multi-Tenant Cloud Solution

Key Terms:

Term	Full Name	What it does
SaaS	Software as a Service	Cloud-hosted software delivered over internet
Tenant	Customer/Organization	A single customer sharing the platform with others
RLS	Row-Level Security	Database feature filtering rows by tenant automatically
JWT	JSON Web Token	Signed token containing user/tenant identity claims
Noisy Neighbor	Resource contention	One tenant consuming resources affecting others

Problem Statement

Design a SaaS platform that serves multiple customers (tenants) with data isolation, customization, and efficient resource utilization.

Clarifying Questions to Ask

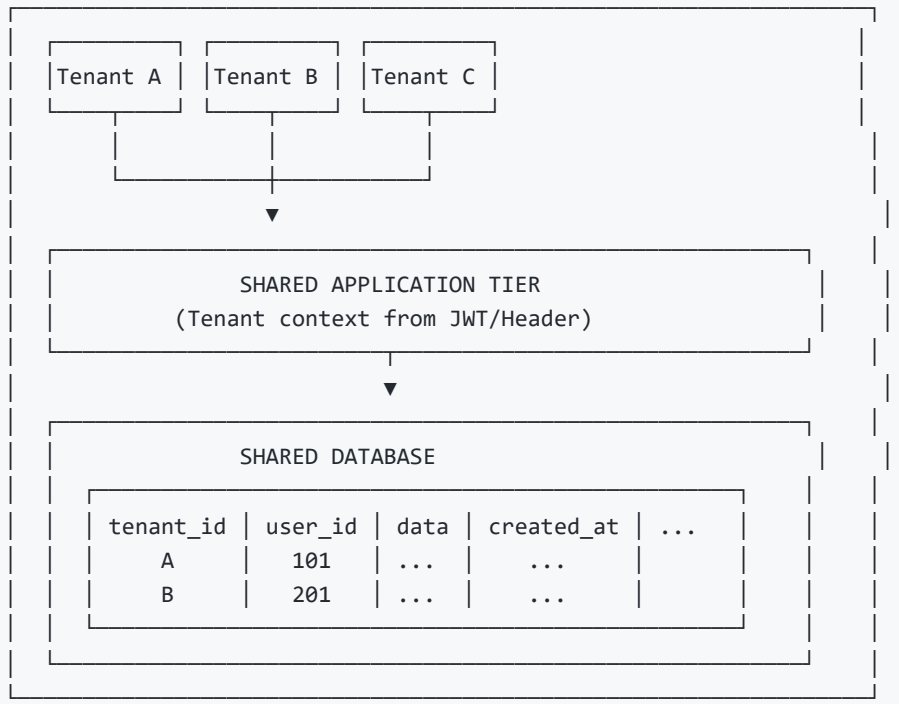
1. **Tenant count**: How many tenants? (100s, 1000s, millions?)
2. **Tenant size variance**: Similar sizes or huge variance?
3. **Isolation requirements**: Strict data isolation or shared acceptable?
4. **Customization**: Do tenants need custom features/branding?
5. **Compliance**: Different compliance needs per tenant?
6. **Pricing model**: Per-user, per-feature, usage-based?
7. **Noisy neighbor**: How to prevent one tenant affecting others?

High-Level Design

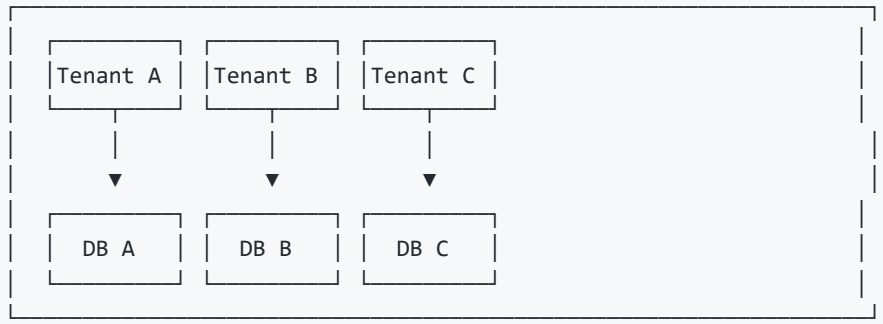
```
┌─────────────────────────────────────────────────────────────────┐
│                    MULTI-TENANT ARCHITECTURE                    │
```



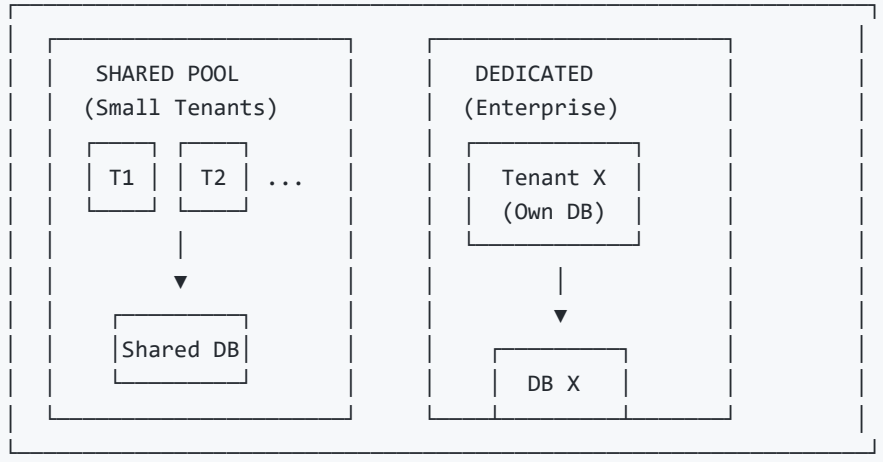
OPTION 1: SHARED DATABASE, SHARED SCHEMA (Cost-effective)



OPTION 2: DATABASE PER TENANT (Better isolation)



OPTION 3: HYBRID (Enterprise tenants get dedicated)



API Design

Tenant Management APIs

Method	Endpoint	Description
`POST`	`/api/v1/tenants`	Provision a new tenant
`GET`	`/api/v1/tenants/{tenantId}`	Get tenant configuration
`PUT`	`/api/v1/tenants/{tenantId}/tier`	Update tenant tier (Basic/Standard/Premium)
`DELETE`	`/api/v1/tenants/{tenantId}`	Deprovision a tenant

Tenant-Scoped Resource APIs

Method	Endpoint	Description
`GET`	`/api/v1/{tenantId}/resources`	List tenant resources
`POST`	`/api/v1/{tenantId}/resources`	Create resource within tenant
`GET`	`/api/v1/{tenantId}/usage`	Get tenant usage metrics

All requests include the `X-Tenant-ID` header for tenant isolation. The API gateway validates tenant context and routes to the appropriate data partition.

Key Components

Component	Purpose	Implementation
Tenant Resolver	Identify tenant from request	Subdomain, header, or JWT claim
Data Isolation	Prevent cross-tenant access	Row-level security or separate DBs
Resource Quotas	Prevent noisy neighbor	Rate limiting per tenant
Config Service	Tenant-specific settings	Feature flags, branding
Billing Service	Usage tracking	Metering and invoicing

Scalability Considerations

- **Tenant sharding**: Distribute tenants across database shards
- **Connection pooling**: Share DB connections across tenants
- **Caching**: Tenant-aware cache keys (tenant:resource:id)
- **Queue isolation**: Separate queues for large tenants

- **Compute isolation**: Dedicated pods for enterprise tenants

Trade-offs

Approach	Pros	Cons	Best For
Shared DB + Schema	Cost-effective, simple	Less isolation	Many small tenants
Schema per Tenant	Good isolation	Migration complexity	Medium tenants
DB per Tenant	Best isolation	Expensive, complex	Enterprise/compliance
Hybrid	Flexible	Complex routing	Mixed tenant sizes

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Cosmos DB	Multi-tenant NoSQL database with partition keys per tenant	Tech Guide
Redis	Tenant-aware caching with prefixed keys (tenant:resource:id)	Tech Guide
AKS/Kubernetes	Compute isolation with dedicated pods for enterprise tenants	Tech Guide
Consistent Hashing	Distribute tenants across database shards evenly	Tech Guide
Rate Limiter	Per-tenant rate limiting to prevent noisy neighbor issues	Tech Guide

How to Explain It Well

> **Opening**: "Multi-tenancy is about serving multiple customers from shared infrastructure while ensuring isolation. I'll discuss three approaches and recommend a hybrid model."

> **Key talking points**:
>
> - "Tenant identification via subdomain (acme.service.com) or JWT claims"

> - "Row-level security in shared DB: every query includes tenant_id filter"
> - "Noisy neighbor prevention: per-tenant rate limits and resource quotas"
> - "Enterprise tenants get dedicated resources for compliance/performance"
> - "Feature flags enable per-tenant customization without code branches"
